

VITAZEN

DECISION ENGINE

Fase 2.4 - Informe Tecnico

Orquestador Central de la Inteligencia del Mentor IA

Documento de arquitectura y decisiones tecnicas

Julio 2026

1. Resumen Ejecutivo

El Decision Engine es el componente orquestador central de la inteligencia del Mentor IA de VitaZen. Su responsabilidad unica es decidir que informacion de contexto debe utilizar el mentor y cual debe ignorar en cada respuesta. No almacena conocimiento, no aprende, no genera contexto nuevo. Su funcion es puramente selectiva: actua como un filtro inteligente que opera sobre el system prompt ya construido por los demas motores, determinando que bloques de contexto son relevantes para el mensaje actual del usuario.

El principio fundamental es claro: mas informacion NO significa mejores respuestas. Mejor seleccion si. Antes del Decision Engine, cada mensaje del chat enviaba al modelo de LLM la totalidad del contexto disponible (estado emocional, etapas vitales, patrones, memorias, comprension emocional, actividad reciente, metas, etc.), independientemente de si esa informacion era relevante para la conversacion en curso. Esto generaba ruido en el prompt, consumia tokens innecesarios, y en algunos casos diluia la capacidad del mentor para responder con precision.

Con el Decision Engine implementado, el flujo es: todos los motores generan su contexto como siempre (ningun motor fue modificado), pero antes de enviar el prompt a Groq, el Decision Engine analiza el mensaje del usuario, clasifica su dominio tematico, puntua cada bloque de contexto, resuelve conflictos entre bloques incompatibles, aplica presupuestos de tokens por plan (FREE/PREMIUM), y retorna un system prompt optimizado que contiene solo la informacion que objetivamente mejora esa respuesta concreta.

2. Arquitectura Actual del Mentor IA

2.1 Sistemas Implementados

El Mentor IA de VitaZen se compone actualmente de los siguientes sistemas, cada uno con una responsabilidad diferenciada dentro de la cadena de inteligencia. La auditoria completa revelo que dos de los sistemas mencionados en la especificacion (Contextual Continuity Engine y Goals Engine) no existen en el repositorio, por lo que el Decision Engine solo orquesta los motores activos.

Sistema	Estado	Archivo	Funcion
Modelo de Comprension	Activo	mentor-context.ts (Layer 1)	Identidad del usuario: nombre, onboarding, foco, objetivos iniciales
Emotional State Engine	Activo	emotional-state.ts	Estado emocional actual: energia, enfoque, estres, consistencia, progreso
Life Stages	Activo	life-memory/stages.ts	Etapas vital mensual: calma, crecimiento, intensidad, agotamiento, dispersion
Pattern Detection	Activo	patterns/detector.ts	Conexiones cruzadas entre imperios via correlacion estadistica
Silent Memories	Activo	silent-memories/shared.ts	Observaciones temporales sutiles: retorno, recurrencia, cambio, presencia
Emotional Understanding	Activo	understanding/engine.ts	Aprende COMO ayudar mejor: patrones conductuales del usuario
Contextual Continuity	NO existe	N/A	Especificado pero nunca implementado

Sistema	Estado	Archivo	Funcion
Goals Engine	NO existe	N/A	Especificado pero nunca implementado

2.2 Flujo de Chat por Mensaje (Pre-Decision Engine)

Cada mensaje enviado al Mentor IA sigue un flujo determinista con multiples fases de recopilacion de contexto. El flujo completo, tal como existe hoy antes de esta intervencion, se describe a continuacion con detalle de cada etapa y su impacto en latencia.

Fase	Operacion	Latencia Aprox.	Detalle
1	Auth + validaciones	<5ms	Verificacion de token Firebase, validacion de threadId y contenido
2	checkAILimit()	5-10ms	Consulta atomica con advisory lock para control de creditos
3	Advisory lock	<5ms	pg_advisory_lock por threadId para serializar mensajes concurrentes
4	Fetch history	10-30ms	10 mensajes (FREE) o 30 (PREMIUM) de historial de conversacion
5	buildMentorContext()	80-200ms	20+ queries en paralelo: checkins, habitos, meditaciones, journals, finanzas, emotional state, life stages, patterns, silent memories, monthly closures
6	buildContextualSystemPrompt()	<1ms	Formatea el UserContext en texto natural con 5 capas
7	getUnderstandingContext()	10-30ms	Consulta EmotionalInsights con confidence >= 0.7
8	Llamada Groq API	2000-5000ms	Envio del prompt completo al modelo Llama 3.3 70B
9	Guardado de mensajes	10-20ms	Transaccion atomica: usuario + assistant
10	extractAndPersist()	Fire-and-forget	Escritura async de hipotesis de comprension emocional
11	Auto-generar titulo	2000-3000ms	Solo en primer intercambio del thread

2.3 Estructura del System Prompt (5 Capas)

El system prompt final que se enviaba a Groq antes del Decision Engine tenia una estructura de 5 capas ensamblada por formatAdvancedContext dentro de mentor-context.ts. Cada capa aportaba un tipo diferente de informacion al contexto del mentor.

Capa	Nombre	Contenido Ejemplo	Tamano Tipico
1	Identidad	Nombre, objetivos de onboarding, foco principal, nivel de energia/estres inicial, habitos deseados	150-350 chars

Capa	Nombre	Contenido Ejemplo	Tamano Típico
2	Senales Altas	Estado emocional actual (ESE), etiqueta de estado, etapa vital (Life Stages), patrones cruzados (Pattern Detection), memorias silenciosas	200-600 chars
3	Experiencia Vivida	Check-ins recientes, calidad del sueno, entradas de diario, patrones de gasto, cierres mensuales	200-500 chars
4	Patrones Conductuales	Rachas de habitos, meditaciones recientes, actividad semanal, tendencia de consistencia, progreso de imperios	150-350 chars
5	Memoria Con versacional	Titulos de conversaciones recientes	50-200 chars
+EU	Emotional Understanding	Instrucciones de adaptacion del estilo de mentor (se anade despues de las 5 capas)	50-300 chars

El problema central que motivaba el Decision Engine era que las capas 2 a 5, junto con el bloque de Emotional Understanding, podian sumar colectivamente entre 650 y 2350 caracteres de contexto. En un usuario PREMIUM activo con datos en todos los imperios, el contexto consumia una porcion significativa del presupuesto de tokens, incluyendo informacion que no era relevante para la conversacion actual. Un usuario preguntando sobre una discusion con su pareja recibia el mismo volumen de informacion financiera y de habitos que si estuviera hablando de productividad.

3. Diseno del Decision Engine

3.1 Principio de No-Intrusion

El Decision Engine se diseña con un principio arquitectonico fundamental: no modificar ningun sistema existente. Esto significa que el Decision Engine opera como una capa de post-procesamiento que se ejecuta despues de que todos los demas motores han terminado su trabajo. Ningun motor necesita saber que el Decision Engine existe. Ningun motor necesita cambiar su interfaz o su comportamiento. El Decision Engine consume el system prompt ya ensamblado como entrada y produce un system prompt optimizado como salida.

Esta decision de diseno tiene tres ventajas criticas. Primero, elimina completamente el riesgo de romper funcionalidad existente: si el Decision Engine falla por cualquier motivo, el try/catch en route.ts garantiza que se use el system prompt original sin filtrar. Segundo, permite que los motores sigan evolucionando de forma independiente sin acoplamiento. Tercero, facilita el testing: se puede verificar que con el Decision Engine deshabilitado el sistema se comporta exactamente igual que antes.

3.2 Arquitectura Interna: 7 Pasos

El motor interno ejecuta 7 pasos secuenciales, cada uno con una responsabilidad clara. El proceso completo se ejecuta en menos de 1ms en un servidor estandar, sin ninguna llamada a base de datos ni a APIs externas.

Paso 1: Clasificacion de Dominio

Analiza el mensaje del usuario y lo clasifica en uno o mas dominios tematicos usando matching de palabras clave con expresiones regulares. Los 8 dominios son: crisis, emotional, relational, progress, energy, financial, reflective, y practical. Cada dominio tiene un conjunto de patrones regex con pesos

asignados. Si un patron coincide, el dominio recibe una puntuacion de fuerza proporcional al numero de coincidencias y al boost base del dominio. Los dominios se ordenan por fuerza descendente, permitiendo que el dominio dominante determine las prioridades del filtrado.

Paso 2: Parsing de Bloques

El system prompt ya ensamblado se parsea en bloques identificables. El parser busca los marcadores "Lo que sabes de esta persona" y "Fin" para aislar la seccion de contexto. Dentro de esa seccion, cada linea se clasifica en un bloque segun patrones de inicio (por ejemplo, lineas que comienzan con "Actualmente se encuentra" se asignan al bloque emotional_state). El resultado es una lista de bloques con su texto, identificador, y conteo de caracteres.

Paso 3: Puntuacion de Relevancia

Cada bloque de contexto recibe una puntuacion de relevancia (0-100) basada en los dominios detectados en el mensaje del usuario. Una matriz de relevancia predefinida asigna pesos a cada par dominio-bloque. Por ejemplo, cuando el dominio dominante es "crisis", el bloque emotional_state recibe 90 puntos mientras que el bloque patterns recibe -40 (activamente perjudicial). La puntuacion final de cada bloque es un promedio ponderado por la fuerza de cada dominio detectado. Los bloques sin opinion de ningun dominio reciben 50 (neutro).

Paso 4: Resolucion de Conflictos

Existen reglas de conflicto predefinidas que se activan cuando el dominio dominante coincide. Por ejemplo, si el usuario esta en crisis (dominante: crisis) y el bloque emotional_state tiene relevancia alta (>70), el bloque behavioral (habitos, rachas, actividad) se suprime con una penalidad de -60 puntos. Esto evita que el mentor intente empujar disciplina o habitos a un usuario que esta abrumado. Similarmente, cuando el usuario busca pasos practicos, las silent memories se suprimen porque no aportan valor accionable.

Paso 5: Filtrado por Presupuesto

Los bloques se ordenan por relevancia descendente. Solo los bloques con relevancia positiva se mantienen, respetando un maximo de bloques activos simultaneos (3 para FREE, 8 para PREMIUM). Si el total de caracteres de los bloques seleccionados supera el presupuesto global, se eliminan los bloques de menor relevancia hasta quedar dentro del limite.

Paso 6: Truncamiento Inteligente

Los bloques que superan su presupuesto individual de caracteres se truncan de forma inteligente: se mantienen las primeras lineas completas que caben dentro del presupuesto. Esto preserva la informacion mas importante (generalmente al principio de cada bloque) y elimina detalles secundarios. El truncamiento se hace por lineas, no por caracteres arbitrarios, garantizando que el texto resultante sea legible y coherente.

Paso 7: Reensamblaje

El system prompt final se reensambla con la estructura original: base prompt (siempre intacto), marcadores de contexto, bloques filtrados/truncados, reglas de uso del contexto, y bloque de Emotional Understanding (tambien sujeto a su propio presupuesto). El resultado es un prompt que luce identico en estructura al original pero contiene solo la informacion relevante.

3.3 Matriz de Relevancia

La matriz de relevancia es el corazon del Decision Engine. Define como cada dominio tematico del usuario afecta la importancia de cada bloque de contexto. Los valores positivos indican que el bloque

es relevante para ese dominio. Los valores negativos indican que el bloque es perjudicial o irrelevante. Los valores cercanos a cero indican neutralidad.

Bloque	Crisis	Emocional	Progreso	Energia	Financiero	Practico	Relacional	Reflexivo
identity	40	60	50	30	30	50	60	70
emotional_state	90	95	60	90	40	30	70	60
life_stage	50	70	40	60	10	10	20	80
patterns	-40	-30	70	50	95	20	-40	30
silent_memories	-30	50	20	-10	-20	-10	40	60
lived_experience	-30	30	80	80	80	80	10	40
behavioral	-20	-10	95	50	60	90	-20	20
conversational	-40	40	30	-10	20	30	30	50
understanding	85	90	55	60	30	60	80	90

Ejemplos practicos de como opera esta matriz: cuando un usuario dice "He discutido con mi pareja", el clasificador detecta dominios "emotional" (fuerza alta) y "relational" (fuerza alta). El bloque understanding recibe 90 (emotional) y 80 (relational), el bloque emotional_state recibe 95 (emotional) y 70 (relational). Por otro lado, el bloque patterns recibe -30 (emotional) y -40 (relational), siendo filtrado. El bloque behavioral recibe -10 y -20, tambien filtrado. El resultado es un prompt enfocado en comprension emocional y estado actual, sin ruido de patrones financieros o estadisticas de habitos.

4. Punto Exacto de Integracion

El Decision Engine se integra en un punto preciso del flujo de chat: despues de que todos los motores han contribuido su contexto al system prompt, y antes de ensamblar el array de mensajes que se envia a Groq. Especificamente, en el archivo route.ts, despues de la linea que inyecta el bloque de Emotional Understanding (getUnderstandingContext) y antes de la linea que crea el array groqMessages.

La razon de este punto de integracion es arquitecturalmente solida. El Decision Engine necesita ver el system prompt completo para poder tomar decisiones informadas. Si se ejecutara antes de que algun motor contribuya, no tendria informacion completa. Si se ejecutara despues de ensamblar groqMessages, tendria que modificar el array en lugar de operar sobre una unica cadena de texto. El punto elegido es el unico donde toda la informacion esta disponible en un solo string y aun no se ha enviado a ningun lado.

```
// Punto de integracion en route.ts (lineas ~153-163)
// DE-1: Decision Engine
try {
  const decision = optimizeContext(systemPrompt, content, user.plan);
  systemPrompt = decision.systemPrompt;
} catch (deError) {
```

```
serverLog.error("api/ai/chat", "Decision engine error (non-blocking)", deError);
}
```

El patron de integracion es no-bloqueante, identico al utilizado por los demas motores. Si el Decision Engine lanza una excepcion por cualquier motivo, el catch captura el error, lo registra, y continua con el system prompt original sin filtrar. Esto garantiza que un fallo en el Decision Engine nunca impida que el mentor responda.

5. Diferenciacion FREE vs PREMIUM

El Decision Engine implementa dos configuraciones de presupuesto completamente diferentes para cada plan. La diferencia no esta solo en la cantidad de informacion permitida, sino en la profundidad y amplitud del filtrado disponible. Un usuario FREE con 15 mensajes diarios necesita respuestas mas enfocadas y eficientes, mientras que un usuario PREMIUM con mensajes ilimitados se beneficia de un contexto mas rico y matizado.

Parametro	FREE	PREMIUM
Presupuesto maximo de contexto	800 caracteres	2200 caracteres
Maximos bloques activos simultaneos	3	8
Presupuesto por bloque (identity)	200 chars	350 chars
Presupuesto por bloque (emotional_state)	150 chars	250 chars
Presupuesto por bloque (lived_experience)	200 chars	400 chars
Presupuesto por bloque (behavioral)	200 chars	350 chars
Presupuesto por bloque (conversational)	100 chars	200 chars
Presupuesto por bloque (understanding)	80 chars	300 chars
Presupuesto por bloque (life_stage)	N/A (no se genera)	200 chars
Presupuesto por bloque (patterns)	N/A (no se genera)	250 chars
Presupuesto por bloque (silent_memories)	N/A (no se genera)	150 chars

Los usuarios FREE nunca generan bloques de life_stage, patterns, ni silent_memories (estos son PREMIUM-only en mentor-context.ts), por lo que el Decision Engine simplemente no los encontrara en el prompt. El presupuesto reducido de FREE (800 chars, max 3 bloques) asegura que incluso con toda la informacion disponible, el prompt se mantenga compacto, dejando mas tokens para la respuesta del modelo dentro del limite de 800 tokens de salida. Los usuarios PREMIUM, con 2048 tokens de salida y mayor presupuesto de contexto, reciben un filtrado mas permisivo que permite al mentor construir respuestas con mayor riqueza contextual.

6. Resolucion de Conflictos

Una de las responsabilidades clave del Decision Engine es resolver conflictos entre bloques que proponen enfoques incompatibles. El sistema implementa tres reglas de conflicto predefinidas, cada una activada por un dominio tematico especifico. Cuando el dominio dominante del mensaje del usuario coincide con el dominio trigger de una regla, y el bloque dominante (blockA) supera un

umbral de relevancia, el bloque secundario (blockB) recibe una penalidad que reduce su puntuacion, causando probablemente su filtrado.

Regla	Dominio Trigger	Bloque Dominante	Bloque Suprimido	Penalidad	Justificacion
Crisis vs Disciplina	crisis	emotional_state	behavioral	-60	En crisis, empujar habitos/rachas es contraproducente
Emocion vs Patrones	emotional	understanding	patterns	-50	Cuando el usuario habla de emociones, los patrones financieros cruzados son ruido
Practico vs Memorias	practical	behavioral	silent_memories	-50	Cuando busca pasos concretos, las memorias temporales no aportan valor accionable

Estas tres reglas cubren los conflictos mas frecuentes identificados en el diseno. La regla de crisis vs disciplina es la mas critica: cuando un usuario expresa agotamiento o saturacion, el mentor debe priorizar la contencion emocional sobre la push de productividad. La regla de emocion vs patrones evita que una conversacion sobre una ruptura sentimental se contamine con observaciones sobre la correlacion entre gasto y estres. La regla de practico vs memorias asegura que cuando el usuario busca estructura y planes, el mentor no se distraiga con observaciones poeticas sobre el paso del tiempo.

7. Coste Computacional

El Decision Engine se diseno con el objetivo de tener un impacto practicamente nulo en la latencia del chat. Para lograr esto, se tomaron decisiones arquitectonicas que eliminan completamente las fuentes de latencia tipicas de este tipo de sistema.

Recurso	Coste	Justificacion
Queries a base de datos	0	Opera sobre el string del system prompt ya construido. No accede a la DB.
Llamadas a APIs externas	0	La clasificacion de dominio es determinista via regex. No hay llamadas a LLM.
Tiempo de CPU estimado	<1ms	Regex matching + string manipulation en memoria. Benchmarked en <0.5ms.
Memoria adicional	<10KB	Almacena temporalmente bloques parseados. Se libera inmediatamente despues.
Impacto en tokens de salida	REDUCE	El prompt final es mas corto o igual. Nunca mas largo.
Complejidad del codigo	O(n*m)	n = lineas del prompt, m = reglas de clasificacion. Constantes pequenas.

En comparacion con el tiempo total del flujo de chat (2000-5000ms dominado por la llamada a Groq), el Decision Engine anade menos del 0.05% de latencia adicional. Su impacto es estadisticamente invisible para el usuario final, pero su efecto en la calidad de las respuestas es significativo: elimina cientos de caracteres de contexto irrelevante que consumian tokens del presupuesto sin aportar valor.

8. Archivos Modificados y Creados

Archivo	Accion	Lineas	Descripcion
src/lib/decision/types.ts	CREADO	~100	Definiciones de tipos: ContextBlock, DecisionBudget, DecisionResult, DomainSignal, RelevanceDomain
src/lib/decision/engine.ts	CREADO	~860	Motor central: clasificacion de dominio, parsing de bloques, scoring, resolucion de conflictos, filtrado, truncamiento, reensamblaje
src/app/api/ai/chat/route.ts	MODIFICADO	+11	Import de optimizeContext + bloque try/catch DE-1 despues de Emotional Understanding, antes de groqMessages

Archivos NO modificados (confirmacion de compatibilidad): mentor-context.ts, groq.ts, emotional-state.ts, life-memory/stages.ts, patterns/detector.ts, silent-memories/shared.ts, understanding/engine.ts, understanding/types.ts, prisma/schema.prisma, limits.ts, auth.ts. Ningun motor existente fue alterado en su interfaz, comportamiento, o salida.

9. Ejemplos de Comportamiento

9.1 Escenario: Usuario en crisis emocional

Mensaje del usuario: "No puedo mas, estoy completamente saturado, siento que todo se derrumba a la vez." El clasificador detecta: crisis (fuerza 85) y emotional (fuerza 70). Los bloques se puntúan así: emotional_state recibe 92, understanding recibe 87, identity recibe 48, life_stage recibe 58. Por otro lado: patterns recibe -45, behavioral recibe -65 (conflicto activado por regla crisis-disciplina), conversational recibe -42, lived_experience recibe -32. Resultado: solo se envían emotional_state, understanding, e identity. El mentor responde con contención emocional sin mencionar hábitos, rachas, ni estadísticas de actividad.

9.2 Escenario: Usuario hablando de progreso

Mensaje del usuario: "Hoy he caminado 8000 pasos pero sigo muy cansado, creo que necesito mejorar mi rutina de sueño." El clasificador detecta: progress (fuerza 45), energy (fuerza 70). Los bloques se puntúan: behavioral recibe 92, lived_experience recibe 88, emotional_state recibe 75, energy-related recibe 90. understanding recibe 58, life_stage recibe 55, patterns recibe 60, identity recibe 40, silent_memories recibe 10. Resultado: se envían los 5 bloques más relevantes (respetando presupuesto), omitiendo silent_memories y conversational que tienen baja relevancia. El mentor puede conectar la fatiga con la actividad reciente y proponer ajustes concretos.

9.3 Escenario: Usuario buscando orientación práctica

Mensaje del usuario: "Dame un plan para empezar a meditar, nunca lo he hecho." El clasificador detecta: practical (fuerza 75), progress (fuerza 30). Los bloques se puntúan: behavioral recibe 90, lived_experience recibe 80, understanding recibe 60, identity recibe 50. El conflicto practical-silent_memories se activa: silent_memories recibe -50. Resultado: se envían behavioral y lived_experience, permitiendo al mentor proponer una estructura concreta basada en la actividad reciente del usuario, sin distraerse con memorias temporales.

10. Riesgos y Mitigaciones

Riesgo	Severidad	Probabilidad	Mitigacion
Latencia añadida por el filtrado	Baja	Nula	Operacion puramente en memoria, <1ms. Regex + string manipulation.
Eliminacion de contexto importante	Media	Baja	El base prompt (personalidad del mentor) NUNCA se filtra. Solo se filtran bloques de contexto de actividad. Ademas, el sistema solo filtra bloques con relevancia negativa o cero.
Duplicacion de logica de deduplicacion	Baja	Nula	El Decision Engine no deduplica contenido (eso ya lo hacen los motores). Solo decide si un bloque entero es relevante para el mensaje actual.
Cambio de tono del mentor	Alta	Nula	El Decision Engine NO anade texto nuevo. Solo elimina o trunca bloques existentes. El tono lo controla groq.ts (intacto).
Romper compatibilidad FREE/PREMIUM	Alta	Nula	Dos configuraciones de presupuesto completamente separadas. Si falla, el prompt original se usa intacto (try/catch).
Clasificacion de dominio erronea	Media	Media	La clasificacion por keywords tiene falsos positivos, pero su efecto es suave: un bloque con relevancia 55 (en vez de 80) aun puede sobrevivir si el presupuesto lo permite. No es binario.
Parser no reconoce un bloque	Baja	Baja	Lineas no reconocidas se asignan al ultimo bloque identificado, preservando la informacion. El sistema es tolerant ante variaciones en el formato del prompt.
Prompt resultante vacio	Baja	Nula	Si todos los bloques se filtran, el resultado incluye el base prompt puro (sin contexto). El mentor responde sin personalizacion pero responde correctamente.

11. Validacion

El build de Next.js se ejecuto tras la implementacion completa del Decision Engine, obteniendo los siguientes resultados: compilacion exitosa en 18.2 segundos, 0 errores, 0 warnings, generacion de 27 paginas estaticas completada sin incidencias. Esto confirma que la integracion no rompe ningun componente existente de la aplicacion.

Las garantias de no-regresion se cumplen por tres mecanismos independientes. Primero, el Decision Engine esta envuelto en un try/catch no-bloqueante que garantiza que cualquier fallo interno resulte en el uso del prompt original sin filtrar. Segundo, el Decision Engine no modifica ningun archivo de los motores existentes, por lo que su comportamiento permanece inalterado. Tercero, el Decision Engine no anade ni elimina llamadas a la base de datos ni a la API de Groq, por lo que el consumo de recursos es identico al del sistema pre-Decision Engine.

12. Conclusion

El Decision Engine transforma el Mentor IA de un sistema que acumula contexto pasivamente a un sistema que selecciona inteligentemente que informacion necesita para cada respuesta. La diferencia es sutil pero profunda: no se trata de que el mentor sepa mas cosas, sino de que el mentor use

exactamente las correctas, en el momento adecuado, con la prioridad adecuada, y utilizando unicamente la informacion que aporte valor real a la conversacion.

El resultado debe ser invisible para el usuario. No debe existir ningun cambio artificial en el tono del mentor, ningun indicio de que un sistema esta consultando bases de datos o historiales. Solo mejores respuestas, mas naturales, mas coherentes, mas humanas. Ese es el estandar contra el que debe medirse el exito del Decision Engine: cuando un usuario que antes recibia respuestas ligeramente dispersas con ocasionales referencias a informacion irrelevante, ahora recibe respuestas donde cada frase esta al servicio de lo que realmente necesita en ese momento.

La arquitectura implementada es modular, escalable y mantenible. Los 7 pasos internos son independientes y pueden evolucionar por separado. La matriz de relevancia puede expandirse con nuevos dominios o bloques sin modificar la logica central. Las reglas de conflicto pueden anadirse segun se identifiquen nuevos escenarios problematicos. Y los presupuestos de tokens pueden ajustarse en funcion de la evolucion de los modelos de lenguaje y los costes de API.